

추적 및 스코어링 시스템 상세 가이드

1. 개요

이 문서는 Recognizer 시스템의 핵심 로직 중 하나인 **추적(Tracking)**과 **스코어링(Scoring)** 시스템의 상세한 정의와 계산 방식을 설명합니다. `config.yaml`에 정의된 각 파라미터가 실제 코드에서 어떻게 동작하는지 분석하여 제공합니다.

2. 추적기 (Tracker) 정의 및 설정

2.1 추적기의 역할

추적기는 포즈 추정 단계에서 감지된 각 사람(객체)에게 고유한 ID를 부여하고, 프레임이 바뀌어도 동일한 사람을 지속적으로 식별하는 역할을 합니다. 이를 통해 시스템은 특정 인물의 연속적인 움직임과 행동 패턴을 분석할 수 있습니다. 본 프로젝트에서는 **ByteTrack** 알고리즘을 사용합니다.

2.2 tracking 설정 상세 분석 (config.yaml)

`config.yaml`의 `tracking` 섹션은 ByteTrack의 동작을 제어합니다.

```
tracking:
  tracker_name: bytetrack
  frame_rate: 30
  track_high_thresh: 0.4
  track_low_thresh: 0.1
  new_track_thresh: 0.5
  track_buffer: 120
  match_thresh: 0.5
  use_hybrid_matching: true
  iou_weight: 0.7
  keypoint_weight: 0.3
```

- `track_high_thresh: 0.4` : 신뢰도 0.4 이상인 탐지를 '신뢰도 높은(High-Confidence)' 객체로 간주합니다. 이 객체들은 새로운 트랙을 시작할 자격이 있습니다.
- `track_low_thresh: 0.1` : 신뢰도 0.1 미만인 탐지는 노이즈로 간주하여 무시합니다. 0.1과 0.4 사이의 탐지는 '신뢰도 낮은(Low-Confidence)' 객체로, 기존 트랙이 잠시 가려졌을 때 연결하는 용도로만 사용됩니다.
- `new_track_thresh: 0.5` : '신뢰도 높은' 객체 중에서도 신뢰도가 0.5 이상이어야만 새로운 트랙 ID를 부여받습니다.
- `track_buffer: 120` : 객체가 화면에서 사라져도 120프레임(4초 @30fps) 동안 트랙 정보를 유지합니다. 이 시간 안에 다시 나타나면 같은 ID를 부여받아 추적의 연속성을 보장합니다.
- `match_thresh: 0.5` : 두 객체를 동일 객체로 판단하기 위한 유사도 임계값입니다.
- `use_hybrid_matching: true` : `true` 일 경우, 단순 바운딩 박스 겹침(IoU)뿐만 아니라 키포인트 위치 유사도까지 함께 고려하여 매칭 정확도를 높입니다.
- `iou_weight` & `keypoint_weight` : 하이브리드 매칭 시 최종 유사도 점수를 계산하기 위한 가중치입니다.
(IoU 점수 * 0.7) + (키포인트 유사도 * 0.3) 와 같이 계산됩니다.

3. 스코어러 (Scorer) 정의 및 설정

3.1 스코어러의 역할

스코어러는 추적된 각 인물의 시간적 포즈 시퀀스 데이터를 분석하여, 특정 행동(폭력, 쓰러짐 등)이 발생했을 확률을 정량적인 점수로 계산합니다. 이 점수는 이후 이벤트 탐지 단계에서 사용됩니다. 서비스별(`fight` , `falldown`)로 특화된 스코어러가 존재합니다.

3.2 Fight 스코어러 (region_based) 실제 계산 로직

`fight_scorer.py`의 `EnhancedFightInvolvementScorer` 클래스를 기반으로, `config.yaml`의 `scoring.fight.weights`가 각 항목별 점수 계산에 어떻게 적용되는지 설명합니다.

최종 점수 공식:

최종 점수 = (movement × 0.4) + (interaction × 0.4) + (position × 0.1) + (temporal × 0.1)

1. movement (움직임 점수, 가중치 0.4)

- **계산 로직:** `_calculate_movement_intensity()` 함수에서 계산됩니다.
 - i. 연속된 프레임 간 17개 키폰트의 이동 거리를 각각 계산합니다.
 - ii. 이동 거리의 평균 + 2 * 표준편차 를 "급격한 움직임"의 임계값으로 설정합니다.
 - iii. 전체 키폰트 중 이 임계값을 넘는 키폰트의 **비율**을 계산합니다.
 - iv. 추적된 기간 동안 이 비율의 평균값이 최종 점수가 됩니다.
- **의미:** **신체 부위 중 몇 퍼센트가 평균보다 훨씬 빠르게 움직이는가**를 측정하여 움직임의 격렬함을 평가합니다.

2. interaction (상호작용 점수, 가중치 0.4)

- **계산 로직:** `_calculate_enhanced_interaction()` 함수에서 계산됩니다.
 - i. 분석 대상과 다른 모든 사람 간의 상호작용을 프레임별로 계산합니다.
 - ii. 각 프레임의 상호작용 점수는 **근접도 점수**와 **움직임 동기화 점수**의 평균으로 결정됩니다.
 - **근접도 점수:** 두 사람의 중심점 간 거리를 각자의 크기로 정규화하여 계산합니다. 거리가 가까울수록 1에 가까운 높은 점수를 받습니다.
 - **움직임 동기화 점수:** 두 사람의 움직임 벡터(방향과 크기) 간의 **코사인 유사도**를 계산합니다. 서로를 향해 움직이거나 같은 방향으로 움직이는 등, 움직임이 연관될수록 점수가 높아집니다.
 - iii. 추적된 기간 동안 이 상호작용 점수의 평균값이 최종 점수가 됩니다.
- **의미:** **사람들이 얼마나 가깝게 붙어 있으며, 움직임이 서로 얼마나 연관되어 있는가**를 측정합니다.

3. position (위치 점수, 가중치 0.1)

- **계산 로직:** `RegionBasedPositionScorer.calculate_position_score()` 함수에서 계산됩니다.
 - i. 화면을 중앙과 네 모서리의 5개 영역으로 나눕니다. 각 영역은 중요도 가중치(중앙: 1.0, 가장자리: 0.7~0.8)를 가집니다.
 - ii. 한 사람의 전체 경로를 따라 각 영역에서의 활동도를 누적합니다.
 - iii. (영역별 활동도 / 전체 프레임) * 영역 가중치 를 계산하여 5개 영역의 점수를 구합니다.
 - iv. 이 5개 점수 중 **가장 높은 값**이 최종 점수가 됩니다.
- **의미:** **가장 활발하게 활동한 영역의 중요도**를 점수화합니다.

4. temporal (시간적 연속성 점수, 가중치 0.1)

- **계산 로직:** `_calculate_temporal_consistency()` 함수에서 계산됩니다.
 - i. 연속된 세 프레임(t-2, t-1, t)을 기준으로, (t-2)--(t-1) 움직임 벡터와 (t-1)--(t) 움직임 벡터를 구합니다.
 - ii. 두 벡터 간의 **코사인 유사도**를 계산하여 움직임 방향의 일관성을 측정합니다.
 - iii. 추적된 기간 동안 이 유사도 점수들의 평균값이 최종 점수가 됩니다.
- **의미:** **움직임의 방향이 얼마나 일관적인가**를 측정합니다. 점수가 낮을수록 방향 전환이 잦고 예측 불가능한 움직임을 의미할 수 있습니다.

※ 고립 패널티 (isolation_penalty)

- **로직:** 한 사람이 다른 사람들과 계속 멀리 떨어져 '고립' 상태로 판단되면, 위에서 계산된 최종 폭력 점수가 아무리 높아도 **강제로 0.1점 이하로 깎입니다**. 싸움은 혼자 할 수 없다는 물리적 제약을 반영한 로직입니다.

3.3 Falldown 스코어러 (falldown_scorer) 실제 계산 로직

`falldown_scorer.py` 의 `FalldownScorer` 클래스를 기반으로, `config.yaml` 의 `scoring.falldown.weights` 가 각 항목별 점수 계산에 어떻게 적용되는지 설명합니다.

최종 점수 공식:

$$\text{최종 점수} = (\text{height_change} \times 0.35) + (\text{posture_angle} \times 0.25) + (\text{movement_intensity} \times 0.20) + (\text{persistence} \times 0.15) + (\text{position} \times 0.05)$$

1. height_change (높이 변화 점수, 가중치 0.35)

- **계산 로직:** `calculate_height_change_score()` 함수에서 계산됩니다.
 - i. 추적 기간 동안 각 프레임의 바운딩 박스 높이를 계산합니다.

- ii. 연속된 프레임 간 높이 **감소 비율**을 계산합니다.
- iii. 전체 기간 중 **가장 큰 단일 감소 비율**(`max_height_drop`)을 찾습니다.
- iv. 이 값이 40%(`height_drop_threshold`) 이상이면 높은 점수를, 미만이면 낮은 점수를 부여합니다.
- **의미:** ****순간적으로 사람의 키가 얼마나 급격하게 줄어들었는가****를 측정합니다. 쓰러짐의 가장 핵심적인 지표입니다.

2. posture_angle (자세 각도 점수, 가중치 0.25)

- **계산 로직:** `calculate_posture_angle_score()` 함수에서 계산됩니다.
 - i. 각 프레임에서 '어깨 중심'과 '골반 중심'을 잇는 몸의 중심축 각도를 계산합니다.
 - ii. 이 각도가 수직(서 있는 상태)에서 45도(`angle_threshold`) 이상 기울어지면 점수를 부여하기 시작하며, 90도(수평 상태)에 가까울수록 1점에 가까운 높은 점수를 받습니다.
 - iii. 추적된 기간 동안 이 프레임별 점수들의 **평균값**이 최종 점수가 됩니다.
- **의미:** ****평균적으로 얼마나 수평으로 누운 자세에 가까웠는가****를 측정합니다.

3. movement_intensity (움직임 강도 점수, 가중치 0.20)

- **계산 로직:** `calculate_movement_intensity_score()` 함수에서 계산됩니다.
 - i. 연속된 프레임 간 바운딩 박스 중심점의 이동 거리를 계산합니다.
 - ii. 전체 기간 중 **가장 큰 단일 이동 거리**(`max_movement`)를 찾습니다.
 - iii. 이 값이 50픽셀(`movement_threshold`)을 초과하면 높은 점수를 받습니다.
- **의미:** ****가장 격렬했던 한순간의 움직임****을 측정하여, 쓰러지기 직전의 휘청임이나 넘어지는 순간의 큰 동작을 포착합니다.

4. persistence (지속성 점수, 가중치 0.15)

- **계산 로직:** `calculate_falldown_persistence_score()` 함수에서 계산됩니다.
 - i. 각 프레임마다 쓰러짐 상태를 나타내는 여러 지표(바운딩 박스 가로/세로 비율, 몸 기울기, 화면 내 위치)를 확인하여 '프레임별 쓰러짐 점수'를 계산합니다.
 - ii. 이 점수들의 **평균값**을 기본 점수로 합니다.
 - iii. 여기에 '프레임별 쓰러짐 점수'가 0.5 이상인 프레임이 **연속된 길이**에 비례하는 **보너스 점수**(최대 0.3점)를 더합니다.
- **의미:** ****쓰러진 상태가 얼마나 오래, 그리고 얼마나 연속적으로 지속되었는가****를 측정합니다.

5. position (위치 점수, 가중치 0.05)

- **계산 로직:** `calculate_position_score()` 함수에서 계산됩니다.
 - i. 사람의 위치가 화면 정중앙에 가까울수록 높은 점수를 받습니다.
 - ii. 추적 기간 동안의 평균 점수를 사용합니다.
- **의미:** 쓰러짐 이벤트 자체와는 연관성이 낮아 가중치가 매우 낮게 설정되어 있습니다.